

pytest (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-pytest>  <https://github.com/learn-co-curriculum/python-p3-pytest/issues/new>

Learning Goals

- Explain how pytest is used to ensure our code works as expected.
 - Configure an application to run pytest.
 - Describe the structure of a pytest test.
 - Execute pytest with flags to ensure its output is what we need.
 - Interpret pytest output to find the sources of errors.
-

Key Vocab

- **Unit Testing:** a development process where the smallest testable parts of an application are independently tested for proper functionality.
 - **Test-Driven Development:** a development process where tests are written to meet expectations for an application before code is written to meet those expectations.
 - **Assertion:** a statement that determines if a wrapped statement produces a falsy value or exception. In either case, the assertion fails and the execution of code stops.
 - **Flag:** a method of providing options to modify commands from the command line. Flags begin with a dash - .
-

Introduction

pytest is a testing framework in Python that makes it easy to write short, easy-to-parse tests for applications ranging from a single function to huge libraries.

pytest Helps You Write Better Programs

pytest is primarily used for a task called **unit testing**. Unit testing is a process in which the smallest parts of an application- no matter how large the application- are looked at individually and tested to make sure they operate as intended. Testing is usually carried out by developers themselves, but sometimes by teams of quality assurance (QA) engineers as well.

Unit tests will typically run on many functions at once with many different types of input. The number of failures with their descriptions will be returned to the tester so that they can update any non-functional code. Tests can be run quickly and as many times as desired. They can also be run on code that doesn't exist yet- this allows for a process called **test-driven development** (TDD), where failures are used to inform what code to write next.

An Example of TDD

Let's say you want to write a function that interpolates into a string: given a name, it should return "Welcome, name!"

This is fairly simple to write, but even easier to test. Let's start by writing an assertion that will raise an Exception if the return value of our function is incorrect. Open up the Python shell and enter the following:

```
>>> def interpolate_welcome(name):  
...     pass  
...  
>>> assert interpolate_welcome('Guido') == 'Welcome, Guido!'
```

You should see the following output:

```
# => Traceback (most recent call last):  
# =>   File "<stdin>", line 1, in <module>  
# => AssertionError
```

This tells us that in the most recent block of code (the assertion), there was an error on the first line. As this is an assertion, the error is easy enough to parse out: `interpolate_welcome()` doesn't interpolate the name we pass in!

Let's use that assertion to drive the development of some working code:

```
>>> def interpolate_welcome(name):  
...     return f'Welcome, {name}!'  
...  
>>> assert interpolate_welcome('Guido') == 'Welcome, Guido!'
```

You should see no output from this assertion- that means that it worked!

Simple assertions are *enough* for us to do TDD, but they don't give us many details. We'll see how pytest improves upon this process in just a bit.

What Does a pytest Test Look Like?

While you've worked with several pytest tests up to this point, you may not have dug into the test files themselves. Let's do that now.

pytest File Structure

The first thing that we need to do to create an application environment that supports pytest is include either `pytest.ini` or `setup.py`. pytest generates its own paths when it is run, and can often struggle to find the files that you wish to test. The inclusion of these files allows us to specify where pytest should start building paths. We give it two options: `.`, the root directory, and `lib`, the application directory.

Once this is set up, we should create a directory to house our tests. This can be named any valid Python package name (no dashes!), but we recommend making it clear that it contains test. Ours is contained in `lib/` and is simply called `testing/`.

We've put a couple of tests inside of `testing`: `test_string.py` and `subdirectory/bool_test.py`. pytest files must be named either starting with `"test_"` or ending with `"_test"`. pytest will look in the current directory and every subdirectory for any files that match this naming pattern and execute any tests within.

pytest Test Structure

The test themselves have to be named fairly strictly: `test_{name}` for functions and `Test{Group}` for classes. We'll explain what this means a bit more later on in Phase 3- just know that pytest classes contain groups of tests and pytest functions are single tests.

Running pytest

pytest can be executed from the command line using the command `pytest`. This will run every test in the current directory and any subdirectories, with paths to separate files being determined by `pytest.ini`. So long as you execute pytest from one of the directories specified there, you shouldn't have any issues getting your tests to run.

Let's start off by simply running `pytest`. Enter your virtual environment from the project root directory with `pipenv install; pipenv shell` and enter the command `pytest`:

```
$ pytest
```

```
===== test session starts =====
platform darwin -- Python 3.8.13, pytest-7.2.1, pluggy-1.0.0
rootdir: python-p3-pytest, configfile: pytest.ini
collected 3 items
```

```
string_functions.py contains a function "return_string()" that returns a variable of type str. .
string_functions.py contains a function "interpolate_string()" that takes a string and inserts it into another string
string_functions.py contains a function "return_true" that returns True. F
```

```
===== FAILURES =====
```

```
_____ test_return_true _____
```

```
def test_return_true(self):
    '''contains a function "return_true" that returns True.'''
> assert return_true() == True
E assert False == True
E + where False = return_true()
```

```
lib/testing/subdirectory/bool_test.py:10: AssertionError
===== short test summary info =====
FAILED string_functions.py contains a function "return_true" that returns True. - assert False == True
```

There's a lot to parse here! Rather than jumping right into this output, let's first figure out how to tailor our output to our needs. We only failed one test, after all- how can we run the one that failed?

Specifying Tests to Run

pytest provides a number of different options to specify which tests to run. We can run all tests, all tests in a directory, all tests in a file, all tests in a class, and only specified tests.

Running All Tests

As we saw before, running all tests just requires us to run `pytest` from the project root directory. If all tests are in a subdirectory (such as `lib/` or `testing/` in this repo), we can also run `pytest` from there.

Running All Tests in a Directory

An easier way to run all tests in a directory is to specify the directory after the `pytest` command:

```
$ pytest lib/testing
(python-p3-pytest) python-p3-pytest % pytest lib/testing
===== test session starts =====
...
module in bool_functions, function "return_true" returns True. - assert False == True
===== 1 failed, 2 passed in 0.05s =====
```

Running All Tests in a File

We can see above that only one test is failing: `test_return_true()`. This is contained in `bool_test.py`. We can specify to run the tests in this file with the same syntax as above, noting that `bool_test.py` is contained in a subdirectory of `testing/`: `pytest`

```
lib/testing/subdirectory/bool_test.py .
```

```
$ pytest lib/testing/subdirectory/bool_test.py
===== test session starts =====
...
===== short test summary info =====
FAILED module in bool_functions, function "return_true" returns True. - assert False == True
===== 1 failed in 0.04s =====
```

Running One Test in a File

Getting bored of the same syntax? Good news! Navigating the contents of a file has nothing to do with Unix directory structure, so we won't be adding any more forward slashes `/`.

pytest uses a double colon `::` to navigate from the file itself down to classes and functions. Since we don't have any classes in our tests, we can run `pytest lib/testing/bool_test.py::test_return_true` to look into this error specifically.

```
$ pytest lib/testing/subdirectory/bool_test.py::test_return_true
===== test session starts =====
...
===== short test summary info =====
FAILED module in bool_functions, function "return_true" returns True. - assert False == True
===== 1 failed in 0.04s =====
```

Now that we can run the tests that we want, let's start parsing useful information out of the results.

Interpreting pytest Output

Let's go back and look at the *full* output from the `pytest` command:

```
===== test session starts =====
```

```
platform darwin -- Python 3.8.13, pytest-7.2.1, pluggy-1.0.0
```

```
rootdir: python-p3-pytest, configfile: pytest.ini
```

```
collected 3 items
```

```
string_functions.py contains a function "return_string()" that returns a variable of type str. .
```

```
string_functions.py contains a function "interpolate_string()" that takes a string and inserts it into another string
```

```
string_functions.py contains a function "return_true" that returns True. F
```

```
===== FAILURES =====
```

```
_____ test_return_true _____
```

```
def test_return_true():
```

```
    '''in bool_functions, function "return_true" returns True.'''
```

```
> assert return_true() == True
```

```
E assert False == True
```

```
E + where False = return_true()
```

```
lib/testing/subdirectory/bool_test.py:7: AssertionError
```

```
===== short test summary info =====
```

```
FAILED string_functions.py contains a function "return_true" that returns True. - assert False == True
```

```
===== 1 failed, 2 passed in 0.05s =====
```

There's a lot to look through here- let's go line by line.

```
===== test session starts =====
```

```
platform darwin -- Python 3.8.13, pytest-7.2.1, pluggy-1.0.0
```

```
rootdir: python-p3-pytest, configfile: pytest.ini
```

```
collected 3 items
```

We begin with a message that the test session has started, with some equals signs `=` building a border. This is to help you find the test results if you need to scroll back at any point. The next two lines simply describe your configuration for your system and pytest- the Python version, the pytest version, the root directory, configuration file, and so on. "collected 3 items" tells us that pytest has found three tests.

```
string_functions.py contains a function "return_string()" that returns a variable of type str. .
string_functions.py contains a function "interpolate_string()" that takes a string and inserts it into another string
string_functions.py contains a function "return_true" that returns True. F
```

Next, pytest shows us the progress of testing and whether each test passed or failed. `.` after the test description denotes a pass, `F` denotes a failure. The percentages in brackets inform us of how far we have progressed in testing- this can be useful if certain functions in your application take a long time to run.

```
===== FAILURES =====
_____ test_return_true _____

    def test_return_true():
        '''in bool_functions, function "return_true" returns True.'''
>       assert return_true() == True
E       assert False == True
E       + where False = return_true()
```

```
lib/testing/subdirectory/bool_test.py:7: AssertionError
```

After looking at an overview of the tests, we dig deeper into the failures. pytest begins with the name of the test that failed, then shows the code from which the error arose. Along the left, look for an arrow (greater than) symbol `>`. This shows the specific line of code that produced an error when interpreted.

Underneath the line of code that produced the error, we see lines that begin with an "E". The first "E" line will show the values of the interpreted line of code- here, we can see that `False` is being asserted to equal `True`. Naturally, this is cause for an error. The next line will show us where this erroneous value came from- `False` is the return value of `return_true()`. Finally, pytest provides the path to the line of the error from the location the test was run. `lib/testing/subdirectory/bool_test.py` on line 7 produced an `AssertionError`.


```
===== short test summary info =====  
FAILED string_functions.py contains a function "return_true" that returns True. - assert False == True  
===== 1 failed, 2 passed in 0.05s =====
```

At the end of every testing session, pytest provides an overview of the important takeaways for developers and QA engineers. Inside of this summary, the description of the test is included alongside a note that it failed and the interpreted code that caused the failure. We then get the number of failures, number of passed tests, and the amount of time it took to run all tests.

Note: While we won't be looking at the times during our tests, these are going to be important to pay attention to when building larger applications. Slower applications mean fewer users- you don't want that!

Customizing pytest Output

When running commands from the command line, you often have the option to include **flags**. Flags are a way to modify your commands- they begin with a dash `-`, and they're tailored to each command they're used with. For example, `cp` (copy) has a `-r` flag that specifies that the copy operation should be recursive- you need to use this to copy a directory and its contents. `tree` has a `-L` flag that allows you to specify the depth of subdirectories you want to include in its output.

pytest has *many* flags, but we're just going to focus on the few that will be most helpful to you.

- `-x` is pytest's "exit" flag. This executes tests until one fails, then stops executing tests. This is very helpful for test-driven development, as you'll want to focus on developing to one test at a time.
- `--pdb` opens the Python debugger when a test fails. It does not open the prettier, improved `ipdb`, but its basic functions are very similar.
- `-s` tells pytest to show the full output for failed tests (i.e. `print()` statements).
- `-q` (for quiet) shortens pytest's output. Running with the `-q` flag will only show a single line for the summary of the testing session and details of the failures.
- `-h` will help you figure out where to place arguments for the `pytest` command *and* provide a long list of flags and configurations for use with pytest.

Instructions

This is a test-driven lab. Run `pytest -x` to execute tests until the first fails, then use your new knowledge of pytest and assertions to get all tests passing.

When you're done with this first task, write a test in `testing/test_not_none.py` to help get the function in `not_none_functions.py` to pass.

Submit your work with `git` when all tasks are complete.

Conclusion

You've seen that pytest is a valuable tool with the Python labs you've been working on. Using pytest, we are able to construct tests that help ensure that you understand the concepts and can use them successfully. Well-written tests also help you understand more clearly what the expectations are for what your code will do and how it will function.

But this is not the only reason to learn how to use pytest - it will also be valuable to you in your career as a Python software engineer. Testing packages like pytest are used for test-driven development (TDD), which is considered to be the one of the best processes for writing robust, well-designed, error-free code. Because tests focus on the uses of software rather than the details of its implementation, TDD leads to increased attention to usability and functionality, which in turn results in more stable, usable applications. Finally, having test coverage can make it easy to detect bugs that are introduced during the development process, as well as bugs in existing code.

Think of this lesson as a reference. You should have an understanding of what pytest is and what it does, but you do not need to be a pytest wizard before moving on. As you proceed through the curriculum, if you're ever struggling with figuring out a lab, this is a great place to return to help you get the most out of pytest.

Resources

- [pytest - helps you write better programs](https://docs.pytest.org/en/7.2.x/)  [\(https://docs.pytest.org/en/7.2.x/\)](https://docs.pytest.org/en/7.2.x/)
- [Command-line Flags - pytest](https://docs.pytest.org/en/7.1.x/reference/reference.html#command-line-flags)  [\(https://docs.pytest.org/en/7.1.x/reference/reference.html#command-line-flags\)](https://docs.pytest.org/en/7.1.x/reference/reference.html#command-line-flags)

- [Python's assert: Debug and Test Your Code Like a Pro - RealPython](https://realpython.com/python-assert-statement/#:%7E:text=in%20your%20code.,What%20Are%20Assertions%3F,while%20you're%20debugging%20code.)  [\(https://realpython.com/python-assert-statement/#:%7E:text=in%20your%20code.,What%20Are%20Assertions%3F,while%20you're%20debugging%20code.\)](https://realpython.com/python-assert-statement/#:%7E:text=in%20your%20code.,What%20Are%20Assertions%3F,while%20you're%20debugging%20code.)
- [What is Test-Driven Development? - testdriven.io](https://testdriven.io/test-driven-development/)  [\(https://testdriven.io/test-driven-development/\)](https://testdriven.io/test-driven-development/)

This tool needs to be loaded in a new browser window

Load pytest (CodeGrade) in a new window